

Molecular Dynamics Simulation of Helium Atoms

Zhaoyi ZHANG

Table of Contents

1.Introduction

- 1.1 Intermolecular Interactions
- 1.2 Equations of Motion
- 1.3 Verlet Algorithm
- 1.4 Boundary Conditions
- 1.5 Cut-off Radius
- 1.6 Density and Interaction Feature of Rare-gas System

2. Construction of Simulation System

- 2.1 Parameter and Particle-state Initialization
 - 2.1.1 Parameter Initialization
 - 2.1.2 Initial Atomic Positions
 - 2.1.3 Random-number Seed
 - 2.1.4 Initial-velocity Generation and Temperature Scaling
 - 2.1.5 Initialization of Historical-position Array
- 2.2 Main Module
 - 2.2.1 Parameter Initialization
 - 2.2.2 Initial Atomic Positions
 - 2.2.3 Update Velocity, Acceleration and Positions
 - 2.2.4 Temperature-control Strategy
 - 2.2.5 Remove Drift Velocity and Compute Energy
 - 2.2.6 Heat-capacity Calculation and Result Output

3. Results and Analysis

- 3.1 Average system energy $\langle E \rangle$ at $T=300\text{K}$
- 3.2 Constant-volume heat capacity C_V over the temperature range 200 K-400 K
 - 3.2.1 Theoretical heat-capacity value
 - 3.2.2 Computed heat-capacity results
 - 3.2.3 Error Analysis
- 3.3 Parameter Sensitivity Analysis
 - 3.3.1 Sensitivity to Time Step Δt
 - 3.3.2 Sensitivity of total simulation steps n_{step}

4. Conclusion

- 4.1 Main Research Findings and Reliability Analysis
- 4.2 Limitations of This Study
- 4.3 Directions for Code Optimization

1.Introduction

In this project, I use molecular dynamics to study the motion of helium atoms. The system contains 100 helium atoms in a cubic box with a side length of $d=1000 \times 10^{-10}$ m. The mass of a helium atom is given as $4 \times 1.667 \times 10^{-27}$ kg. The atoms interact with each other through the Lennard-Jones potential. There are two main goals in this project. First, I calculate the average energy $\langle E \rangle$ of the system at $T=300$ K. Second, I calculate the constant-volume heat capacity C_V when the temperature ranges from 200 K to 400 K. The complete workflow is shown in Figure 1.

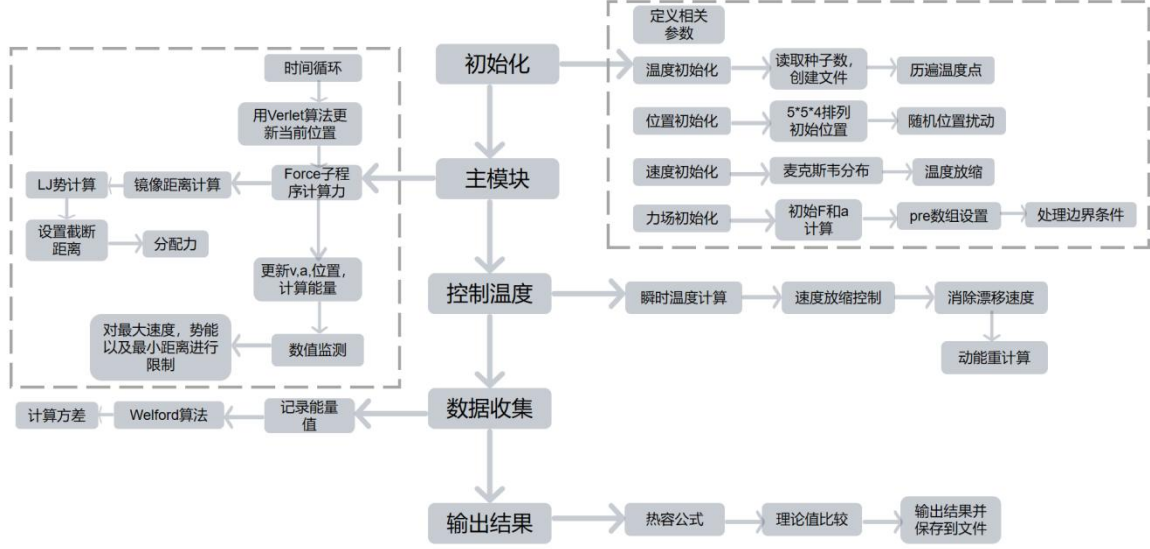


Figure 1. Simulation workflow

1.1 Intermolecular Interactions

The interactions between atoms are modeled using the Lennard-Jones potential. This potential is commonly used to describe the interactions between non-polar molecules. The potential depends on the positions of the atoms and only depends on the distance between two particles, $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$. To simplify the calculation, only pairwise interactions are considered. The Lennard-Jones potential is given by:

$$u(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \quad (1)$$

where σ is the effective diameter of spherical particles. Its value is 1.48×10^{-10} m, which shows the typical length scale for obvious interactions between helium atoms. ϵ describes the strength of attractive interaction between particles. It equals 0.0213 kcal/mol, or 1.48×10^{-22} J in SI units. This value tells us how strong the attraction between helium atoms is. This potential function has two physical parts. The short-range term $(\sigma/r)^{12}$ stands for short-range repulsion. It comes from electron-cloud shielding and the Pauli exclusion principle. Electrons cannot stay in the same quantum state, so strong repulsion appears when atoms get close. The long-range term $(\sigma/r)^6$ represents long-range attraction. It comes from van der Waals dispersion forces, which are caused by interactions between temporary dipoles among molecules.

```

337      ! 计算势能
338      r_inv2=1.0d0/r2
339      r_inv6=r_inv2**3
340      r_inv12=r_inv6**2
341      V_lj=4.0d0*eps1*(sigma1**12*r_inv12-sigma1**6*r_inv6)-V_cut
342      E_pot_out=E_pot_out+V_lj
343

```

Figure 2. Code for potential-energy calculation

Then I can get the total potential energy of the whole system from the formula below:

$$U = \sum_i \sum_{i < j} u(r) \quad (2)$$

Then we take the derivative of potential energy with respect to particle position to get the intermolecular force. This formula can calculate the force on each particle. Its expression is:

$$F = -\sum_{i \neq j} \frac{\partial u(r)}{\partial r} = -24 \sum_{i \neq j} \left[2 \left(\frac{1}{r} \right)^{13} - \left(\frac{1}{r} \right)^7 \right] \quad (3)$$

```

344      ! 计算力
345      f_lj_over_r= 48.0d0*eps1*(sigma1**12*r_inv12-0.5d0*sigma1**6*r_inv6)*r_inv2
346      f_pair = f_lj_over_r

```

Figure 3 the code for force calculation.

1.2 Equations of Motion

The motion of helium atoms follows Newton's second law. The equation of motion for the i -th helium atom is written as:

$$\vec{F}_i = m_i \cdot \vec{a}_i = m_i \cdot \frac{d^2 \vec{r}_i}{dt^2} \quad (4)$$

F_i is the total force acting on the i -th helium atom. The mass of helium atom is $m_i=4 \times 1.667 \times 10^{-27}$ kg. a_i means acceleration, r_i is the position vector of the atom, and t is time. We use the Verlet algorithm to solve this equation approximately.

```

118
119      ! 初始力与加速度
120      call force(xx, yy, zz, fx, fy, fz, E_pot, N, sigma, eps, length, min_r2, 0)
121      accelx=fx/mass
122      accely=fy/mass
123      accelz=fz/mass
124

```

Figure 4 the code for initial-acceleration calculation.

1.3 Verlet Algorithm (Code Line 134-175)

The Verlet algorithm is a common numerical integration method in molecular dynamics. It has time-reversibility and good energy-conservation properties. The algorithm includes these steps:

1.3.1 Position Update

Calculate the positions at the next time step using the positions from the current and previous time steps.

```

140  do atomi = 1, N
141      xxnew(atomi) = 2*xx(atomi) - xxpre(atomi) + accelx(atomi)*dt**2
142      yynew(atomi) = 2*yy(atomi) - yypre(atomi) + accely(atomi)*dt**2
143      zznew(atomi) = 2*zz(atomi) - zzpre(atomi) + accelz(atomi)*dt**2
144      xxnew(atomi) = modulo(xxnew(atomi) + length, length)
145      yynew(atomi) = modulo(yynew(atomi) + length, length)
146      zznew(atomi) = modulo(zznew(atomi) + length, length)
147  end do
148

```

Figure 5. Code for position update

1.3.2 Velocity Calculation

Calculate instantaneous velocity and acceleration from the difference of particle positions.

```

151
152      ! 更新速度，加速度
153  do atomi = 1, N
154      dx=xxnew(atomi)-xxpre(atomi)
155      dx=dx-length*nint(dx/length)
156      vx(atomi)=dx/(2.0d0*dt)
157      dy=yynew(atomi)-yypre(atomi)
158      dy=dy-length*nint(dy/length)
159      vy(atomi)=dy/(2.0d0*dt)
160      dz=zznew(atomi)-zzpre(atomi)
161      dz=dz-length*nint(dz/length)
162      vz(atomi)=dz/(2.0d0*dt)
163      accelx(atomi)=fx(atomi)/mass
164      accely(atomi)=fy(atomi)/mass
165      accelz(atomi)=fz(atomi)/mass
166  end do

```

Figure 6. Code for velocity and acceleration calculation

1.3.3 Historical-Position Update

Update the historical positions for the next time step.

```

167
168      ! 更新位置
169      xxpre = xx
170      yypre = yy
171      zzpre = zz
172      xx = xxnew
173      yy = yynew
174      zz = zznew
175

```

Figure 7. Code for position update

1.4 Boundary Conditions

I use periodic boundary conditions to ignore surface effects. This method creates repeated image boxes to imitate an infinite bulk environment. When a helium atom moves across one wall of the cubic box with side length $d=1000 \times 10^{-10}\text{m}$, it will reappear from the opposite wall. This keeps the atom distribution continuous in the system and makes the simulation closer to real-world conditions.

```

326      ddx=ddx-length1*nint(ddx/length1)
327      ddy=ddy-length1*nint(ddy/length1)
328      ddz=ddz-length1*nint(ddz/length1)

```

Figure 8. Code for periodic mirror boundary conditions

1.5 Cut-off Radius (Used in the force subroutine)

I set the upper limit of cut-off radius to $r=3\sigma$. When the distance between two atoms is larger than this cut-off value, their interaction is ignored. I also set a safety distance $r_{\text{safe}}=0.8\sigma$ to avoid numerical divergence. If r becomes too small, the potential energy U will rise sharply.

```
309      ! 截断以及参数初始化
310      r_cut2=(3.0d0*sigma1)**2
311      r_safe2=(0.8d0*sigma1)**2
---
333      ! 仅处理截断距离内的相互作用
334      if (r2 > r_cut2) cycle
335      if (r2 < r_safe2) r2 = r_safe2 ! 避免数值发散
```

Figure 9. Code for cut-off radius

1.6 Density and Interaction Feature of Rare-gas System

100 helium atoms are placed inside a cubic box with side length $d=1000\times 10^{-10}\text{m}$. The box volume is 10^{-21}m^3 . The total mass of helium atoms is: $m_{\text{total}}=100\times 4\times 1.667\times 10^{-27}\text{kg}\approx 6.668\times 10^{-25}\text{kg}$. The atomic density is:

$$\rho = \frac{m_{\text{total}}}{V} \approx 6.668 \times 10^{-4} \text{ kg / m}^3 \quad (5)$$

This density is much lower than the density of helium gas under standard conditions, which is about 0.1785kg/m^3 . The system belongs to a very dilute gas. The average distance between atoms is quite large, so particle interaction is very weak. According to the physical meaning of the Lennard-Jones potential, obvious interaction only appears when atomic distance is close to the effective diameter $\sigma=1.48\times 10^{-10}\text{m}$. The box side length is $1000\times 10^{-10}\text{m}$. For most atom pairs, their separation is far larger than σ . Therefore, the magnitude of potential energy is naturally very small.

2. Construction of Simulation System

I write this program in Fortran. It contains five main modules: initialization, core integration, force-field calculation, temperature control and result statistics. Data transfer and logical calls between these modules build the whole program structure.

2.1 Parameter and Particle-state Initialization (Code Line 1-133)

This part sets the initial state for 100 helium atoms. First, I set initial positions with a uniform $5\times 5\times 4$ grid. I generate initial velocities with random seed initialization and the Box-Muller transform. I calculate the scaling factor λ , so the initial kinetic energy of the system matches the target temperature of 300 K. I also initialize arrays for historical positions, because the Verlet algorithm needs position data $r(t-\Delta t)$. These steps finish the whole initialization process.

2.1.1 Parameter Initialization (Code Line 1-40)

First input all required parameters. These parameters include fixed values given by the problem and self-defined statistical quantities.

```

1  program final
2      implicit none
3      ! 参数
4      integer, parameter:: N = 100                ! 原子数
5      real*8, parameter:: length = 1.0e-7         ! 边长
6      real*8, parameter:: mass = 6.668e-27        ! He原子质量
7      real*8, parameter:: kb = 1.380649D-23       ! 玻尔兹曼常数
8      real*8, parameter:: pi = 4.0d0*atan(1.0d0)
9      real*8, parameter:: sigma = 1.48e-10        !  $\sigma$ 
10     real*8, parameter:: eps = 1.48e-22          !  $\epsilon$ 
11     real*8, parameter :: dt = 1.0e-16           ! 时间步长
12     integer, parameter :: nsteps = 500000        ! 总模拟步数
13     integer, parameter :: n_eq = 20000          ! 平衡步数
14     integer, parameter :: diagstep = 50000       ! 输出间隔
15     integer, parameter :: r2_diagstep = 10000    ! 粒子间距诊断间隔
16
17     ! 温度数组
18     real*8, dimension(5) :: temp_array = (/200.0d0, 250.0d0, 300.0d0, 350.0d0, 400.0d0/)
19     real*8 :: TT_current ! 当前温度
20     integer :: temp_idx
21
22     ! 核心变量
23     integer :: i, j, k, seedsize, seedd, step, atomi, idx_max, iter
24     real*8 :: xx(N), yy(N), zz(N)                ! 当前位置
25     real*8 :: xxpre(N), yypre(N), zzpre(N)       ! 前一时刻位置
26     real*8 :: xxnew(N), yynew(N), zznew(N)       ! 下一时刻位置
27     real*8 :: vx(N), vy(N), vz(N)               ! 速度
28     real*8 :: fx(N), fy(N), fz(N)               ! 力
29     real*8 :: accelx(N), accely(N), accelz(N)    ! 加速度
30
31     ! 能量与统计变量
32     real*8 :: E_kin, E_pot, E_tot, T_inst
33     real*8 :: meanE, M2, delta, delta2, varE, cv_fluc
34     integer :: n_sample                          ! 采样步数
35
36     real*8 :: min_r2, max_v, tmpv, sum_vx, sum_vy, sum_vz
37     real*8 :: xstep, ystep, zstep, randd, lambda, ll
38     real*8 :: smal, dx, dy, dz
39     integer, allocatable :: seed(:)
40

```

Figure 10. Parameter settings in code

2.1.2 Initial Atomic Positions (Code Line 55-87)

I place 100 helium atoms on a $5 \times 5 \times 4$ three-dimensional grid for even spatial distribution at the start of simulation. I also add small random perturbations to these positions to get more general results.

```

54
55      ! 温度循环
56      do temp_idx = 1, size(temp_array)
57          TT_current = temp_array(temp_idx)
58
59          print "(A,F8.4)", 'temperature: ', TT_current
60
61          ! 重置统计变量
62          meanE = 0.0d0
63          M2 = 0.0d0
64          n_sample = 0
65
66          ! 距离初始化, 考虑缩小原子初始间距
67          smal = 2.0d0 * 2.0d0*sigma
68          xstep=smal/5.0d0
69          ystep=smal/5.0d0
70          zstep=smal/4.0d0
71          atomi = 1
72          do i = 1, 5
73              do j = 1, 5
74                  do k = 1, 4
75                      if (atomi <= N) then
76                          call random_number(randd)      ! 位置随机排列, 提高代码可重复性
77                          xx(atomi)=(length/2.0d0-smal/2.0d0)+(i-1)*xstep+randd*0.1d0*xstep
78                          call random_number(randd)
79                          yy(atomi)=(length/2.0d0-smal/2.0d0)+(j-1)*ystep+randd*0.1d0*ystep
80                          call random_number(randd)
81                          zz(atomi) = (length/2.0d0-smal/2.0d0)+(k-1)*zstep+randd*0.1d0*zstep
82                          atomi = atomi + 1
83                      end if
84                  end do
85              end do
86          end do
87

```

Figure 11. Code for initial atomic positions

This initialization keeps atoms well-separated inside the cubic box without overlap. It avoids unrealistically high energy caused by crowded starting positions. It builds a physically reasonable spatial distribution for the later molecular-dynamics simulation of this dilute gas.

2.1.3 Random-number Seed (Code Line 88-94)

I initialize the seed of the random-number generator to make the simulation repeatable. The random initial velocities in molecular dynamics come from random-number sequences. A fixed seed gives the same initial-velocity distribution in every run. This helps check and reproduce simulation results.

In Fortran, I use the built-in ‘random_seed’ subroutine. It takes an integer value given by users as the random-number seed. The code logic is shown below.

```

87
88      ! 初始化速度
89      call random_seed(size=seedsz)
90      allocate(seed(seedsz))
91      seed = seedd ! 所有温度使用相同的种子
92      call random_seed(put=seed)
93      deallocate(seed)
94

```

Figure 12. Code for random-number seed

2.1.4 Initial-velocity Generation and Temperature Scaling (Code Line 95-110 and 291-300)

Generated initial velocities need to follow the statistical rule of the Maxwell-Boltzmann distribution. I use temperature scaling so the initial kinetic energy matches the target temperature of 300 K. First, I get normally-distributed velocity components v_x , v_y , v_z with the Box-Muller transform. This method turns uniformly-distributed random numbers into normally-distributed

values that fit thermal-motion statistics. Its formula is:

$$v = \sigma \sqrt{-2 \ln U_1} \cos(2\pi U_2) \quad (6)$$

where U_1 and U_2 are uniform random numbers in the range (0,1). σ is the standard deviation of velocity distribution, which depends on temperature. Here I set its value to 1 directly.

```
! Box-Muller 产生标准正态
function box() result(ran)
  real*8 :: ran, u1, u2
  call random_number(u1)
  call random_number(u2)
  ran = sqrt(-2.0d0*log(u1)) * cos(2.0d0*pi*u2)
end function box
```

Figure 13. Schematic diagram of Box-Muller transformation code

After that, I carry out temperature scaling by adjusting factor λ . The relation between initial kinetic energy and temperature follows the equipartition theorem. I use λ to adjust total kinetic energy, so the instantaneous temperature matches the target value of 300 K:

$$\lambda = \sqrt{\frac{3NTk_b}{2E}} \quad (7)$$

Multiply the velocity components of all atoms by λ .

```
94 |
95 | do atomi = 1, N
96 |   ll=box()
97 |   vx(atomi)=ll*sqrt(kb*TT_current/mass)
98 |   ll=box()
99 |   vy(atomi)=ll*sqrt(kb*TT_current/mass)
100 |   ll=box()
101 |   vz(atomi)=ll*sqrt(kb*TT_current/mass)
102 | end do
103 |
104 | ! 速度缩放
105 | E_kin=0.5d0*mass*sum(vx**2+vy**2+vz**2) ! 理论动能
106 | lambda=sqrt((1.5d0*N*TT_current*kb)/E_kin) ! 放缩因子
107 | vx=vx*lambda
108 | vy=vy*lambda
109 | vz=vz*lambda
110 |
```

Figure 14. Code for velocity scaling

2.1.5 Initialization of Historical-position Array (Code Line 119-133)

The integration of the Verlet algorithm needs the position $r(t-\Delta t)$. So I initialize the historical-position array (xxpre, yypr, zzpre) with position $r(t)$, velocity $v(t)$ and acceleration $a(t)$ at $t=0$. The formula is:

$$r(t - \Delta t) = r(t) - v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2 \quad (8)$$

In actual calculation, I first compute the initial acceleration ($a=F/m$). Then I get historical positions from the formula above. I apply periodic boundary conditions to correct positions, so all atoms stay inside the cubic box. The related code is shown below.


```

119 ! 初始力与加速度
120 call force(xx, yy, zz, fx, fy, fz, E_pot, N, sigma, eps, length, min_r2, 0)
121 accelx=fx/mass
122 accely=fy/mass
123 accelz=fz/mass
124
125 do atomi = 1, N
126   xxpre(atomi)=xx(atomi)-vx(atomi)*dt+ 0.5d0*accelx(atomi)*dt**2
127   yypre(atomi)=yy(atomi)-vy(atomi)*dt+ 0.5d0*accely(atomi)*dt**2
128   zzpre(atomi)=zz(atomi)-vz(atomi)*dt+ 0.5d0*accelz(atomi)*dt**2
129   xxpre(atomi)=modulo(xxpre(atomi)+length, length)
130   yypre(atomi)=modulo(yypre(atomi)+length, length)
131   zzpre(atomi)=modulo(zzpre(atomi)+length, length)
132 end do
133

```

Figure 15. Schematic diagram of code for calculating $r(t-\Delta t)$

2.2 Main Module (Code Line 134-370)

The main module is the core of this molecular-dynamics simulation. It includes key steps such as time evolution, force calculation and energy statistics.

2.2.1 Position Update and Boundary-condition Handling (Code Line 134-148)

In each time step, I use the Verlet algorithm to update positions of all atoms. This routine makes atoms re-enter from the opposite box boundary when they cross the wall. It simulates an infinite bulk system.

```

134 ! 上面都是初始化，下面进入正式计算
135 print *, " "
136 print "(A,I6,A,I6)", "Start simulation: Total steps ", nsteps, ", Equilibrium steps", n_eq
137 print *, " "
138 do step = 1, nsteps
139   ! 更新位置
140   do atomi = 1, N
141     xxnew(atomi) = 2*xx(atomi) - xxpre(atomi) + accelx(atomi)*dt**2
142     yynew(atomi) = 2*yy(atomi) - yypre(atomi) + accely(atomi)*dt**2
143     zznew(atomi) = 2*zz(atomi) - zzpre(atomi) + accelz(atomi)*dt**2
144     xxnew(atomi) = modulo(xxnew(atomi) + length, length)
145     yynew(atomi) = modulo(yynew(atomi) + length, length)
146     zznew(atomi) = modulo(zznew(atomi) + length, length)
147   end do
148

```

Figure 16. Schematic diagram of position-update code

2.2.2 Call the force Subroutine to Calculate Forces (Code Line 301-370)

The 'Force' subroutine accurately computes the Lennard-Jones force on each atom and the total potential energy of the system. I use the periodic boundary conditions mentioned in section 1.4 to remove surface effects. When I calculate the interaction between two atoms, I get particle distances within $[-d/2, d/2]$ through the minimum-image convention. Force calculation follows the Lennard-Jones potential function derived in section 1.1.

```

301 subroutine force(xxx, yyy, zzz, ffx, ffy, ffz, E_pot_out, NN, sigma1, eps1, length1, min_r2_out, sstep)
302   implicit none
303   integer, intent(in) :: NN, sstep
304   real*8, intent(in) :: xxx(NN), yyy(NN), zzz(NN), sigma1, eps1, length1
305   real*8, intent(out) :: ffx(NN), ffy(NN), ffz(NN), E_pot_out, min_r2_out
306   integer :: ii, jj
307   real*8 :: ddx, ddy, ddz, r2, r_inv2, r_inv6, r_inv12, V_lj, f_lj_over_r, f_pair, r_cut2, r_safe2, V_cut
308
309   ! 截断以及参数初始化
310   r_cut2=(3.0d0*sigma1)**2
311   r_safe2=(0.8d0*sigma1)**2
312   V_cut=4.0d0*eps1*((sigma1**12/r_cut2**6)-(sigma1**6/r_cut2**3))
313   ffx = 0.0d0
314   ffy = 0.0d0
315   ffz = 0.0d0
316   E_pot_out = 0.0d0
317   min_r2_out = 1.0d99
318
319   ! 计算相互作用
320   do ii = 1, NN-1
321     do jj = ii+1, NN
322       ! 镜像法计算粒子间距
323       ddx=xxx(jj)-xxx(ii)
324       ddy=yyy(jj)-yyy(ii)
325       ddz=zzz(jj)-zzz(ii)
326
327       ddz=zzz(jj)-zzz(ii)
328       ddx=ddx-length1*nint(ddx/length1)
329       ddy=ddy-length1*nint(ddy/length1)
330       ddz=ddz-length1*nint(ddz/length1)
331
332       r2=ddx*ddx+ddy*ddy+ddz*ddz
333       min_r2_out = min(min_r2_out, r2)
334
335       ! 仅处理截断距离内的相互作用
336       if (r2 > r_cut2) cycle
337       if (r2 < r_safe2) r2 = r_safe2 ! 避免数值发散
338
339       ! 计算势能
340       r_inv2=1.0d0/r2
341       r_inv6=r_inv2**3
342       r_inv12=r_inv6**2
343       V_lj=4.0d0*eps1*(sigma1**12*r_inv12-sigma1**6*r_inv6)-V_cut
344       E_pot_out=E_pot_out+V_lj
345
346       ! 计算力
347       f_lj_over_r= 48.0d0*eps1*(sigma1**12*r_inv12-0.5d0*sigma1**6*r_inv6)*r_inv2
348       f_pair = f_lj_over_r
349
350       ! 分配力到两个粒子
351       ffx(ii)=ffx(ii)+f_pair*ddx
352       ffy(ii)=ffx(ii)+f_pair*ddy
353       ffz(ii)=ffz(ii)+f_pair*ddz
354       ffx(jj)=ffx(jj)-f_pair*ddx
355       ffy(jj)=ffx(jj)-f_pair*ddy
356       ffz(jj)=ffz(jj)-f_pair*ddz
357     end do
358   end do
359
360   ! 用来输出最小粒子间距,如果报错可以打开这一部分找出问题在哪一步
361   ! if (mod(sstep, r2_diagstep)==0) then
362   !   print *, "Step ", sstep, " Min r2: ", min_r2_out, " 3sigma*sigma : ", r_cut2
363   ! end if
364
365   ! 检查势能是否稳定
366   if (E_pot_out /= E_pot_out) then
367     print *, "ERROR: NaN potential energy at step", sstep
368     stop
369   end if
370 end subroutine force
371 end program final

```

Figure 17. Schematic diagram of force subroutine code

I add protection and checking mechanisms in this part. Besides the cut-off radius mentioned before, I write code to check whether potential energy stays stable. If something

goes wrong, I can enable lines 358-361 to locate the faulty step. Numerical values may blow up during particle collisions, so these safety checks are quite important.

2.2.3 Update Velocity, Acceleration and Positions (Code Line 152-180)

After force calculation, I compute velocity, acceleration and positions. I also get kinetic energy and total energy at the same time.

```

152      ! 更新速度, 加速度
153      do atomi = 1, N
154          dx=xxnew(atomi)-xxpre(atomi)
155          dx=dx-length*nint(dx/length)
156          vx(atomi)=dx/(2.0d0*dt)
157          dy=yynew(atomi)-yypre(atomi)
158          dy= dy-length*nint(dy/length)
159          vy(atomi)=dy/(2.0d0*dt)
160          dz=zznew(atomi)-zzpre(atomi)
161          dz=dz-length*nint(dz/length)
162          vz(atomi)=dz/(2.0d0*dt)
163          accelx(atomi)=fx(atomi)/mass
164          accely(atomi)=fy(atomi)/mass
165          accelz(atomi)=fz(atomi)/mass
166      end do
167
168      ! 更新位置
169      xxpre = xx
170      yypre = yy
171
172      zzpre = zz
173      xx = xxnew
174      yy = yynew
175      zz = zznew
176
177      ! 计算能量
178      E_kin=0.5d0*mass*sum(vx**2+vy**2+vz**2) ! 动能
179      E_tot=E_kin + E_pot ! 总能量
180      T_inst=(2.0d0*E_kin)/(3.0d0*N*kb)

```

Figure 18. Code for updating particle states

2.2.4 Temperature-control Strategy (Code Line 181-207)

Temperature rises when particles get close or collide with each other. I apply temperature-control strategies to keep the system at a constant target temperature. The whole simulation contains two stages. For steps smaller than `n_eq`, the system stays in the equilibration stage and has not reached equilibrium yet. I use strong velocity scaling to bring the system to target temperature quickly. I set the maximum iteration count to 5 for precise temperature convergence. For steps larger than `n_eq`, the simulation enters the sampling stage. I use mild temperature adjustment. I only modify velocities when temperature deviation exceeds 10 K. In this way, I preserve natural thermal fluctuations of the system and guarantee reliable heat-capacity calculation.

```

181      ! 整个过程中要保持温度在目标温度
182      if (step <= n_eq) then ! 平衡阶段
183          do iter = 1, 5
184              if (abs(T_inst-TT_current)>5.0d0) then
185                  lambda=sqrt(TT_current/T_inst)
186                  vx=vx*lambda
187                  vy=vy*lambda
188                  vz=vz*lambda
189                  ! 重新计算动能和温度
190                  E_kin = 0.5d0*mass*sum(vx**2+vy**2+vz**2)
191                  T_inst=(2.0d0*E_kin)/(3.0d0*N*kb)
192              else
193                  exit
194              end if
195          end do
196      else
197          if (abs(T_inst-TT_current)>10.0d0) then
198              lambda=sqrt(TT_current/T_inst)
199              vx=vx*lambda
200              vy=vy*lambda
201              vz=vz*lambda
202              ! 重新计算动能和温度
203              E_kin=0.5d0*mass*sum(vx**2+vy**2+vz**2)
204              T_inst=(2.0d0*E_kin)/(3.0d0*N*kb)
205          end if
206      end if
207

```

Figure 19. Code for temperature control

2.2.5 Remove Drift Velocity and Compute Energy (Code Line 208-220)

During molecular-dynamics simulation, numerical integration errors, statistical fluctuations of initial velocity distribution and velocity scaling in temperature control may produce net total momentum for the whole system. All atoms may drift together along one certain direction. This drift velocity corresponds to non-zero motion of the system's center-of-mass. Its kinetic energy does not reflect real thermal motion inside the system, but it is counted into total kinetic energy and gives falsely high instantaneous temperature. Therefore I must remove such drift velocity.

```

208      ! 消除漂移速度
209      sum_vx = sum(vx)
210      sum_vy = sum(vy)
211      sum_vz = sum(vz)
212      vx = vx - sum_vx / real(N, 8)
213      vy = vy - sum_vy / real(N, 8)
214      vz = vz - sum_vz / real(N, 8)
215
216      ! 最终动能
217      E_kin=0.5d0*mass*sum(vx**2+vy**2+vz**2)
218      T_inst=(2.0d0*E_kin)/(3.0d0*N*kb)
219      E_tot=E_kin + E_pot
220

```

Figure 20. Code for removing drift velocity

2.2.6 Heat-capacity Calculation and Result Output (Code Line 221-290)

The traditional variance-calculation formula $\langle E^2 \rangle - \langle E \rangle^2$ suffers from severe numerical precision issues. When the magnitude of squared energy is far larger than the variance itself, subtraction of two large numbers causes loss of significant digits and brings large computational errors. The Welford algorithm avoids this problem by recursive updating.

```

221 ! Welford算法
222 if (step > n_eq) then
223     n_sample=n_sample+1
224     delta=E_tot-meanE
225     meanE=meanE+delta/dble(n_sample)
226     delta2=E_tot-meanE
227     M2=M2+delta*delta2
228 end if
229
230 ! 设置最大速度, 防止数值爆炸
231 max_v = 0.0d0
232 idx_max = -1
233 do i = 1, N
234     tmpv = sqrt(vx(i)**2 + vy(i)**2 + vz(i)**2)
235     if (tmpv > max_v) then
236         max_v = tmpv
237         idx_max = i
238     end if
239 end do
240 if (max_v > 1.0d5) then
241     print *, "ERROR: Too large max_v at step", step, "max_v = ", max_v
242     stop
243 end if
244
245 ! 300K的时候每diagstep输出一次中间结果
246 if (TT_current == 300.0d0 .and. mod(step, diagstep) == 0) then
247     write(*, '(I8, 3X, A, F8.2, 3X, A, ES12.5, 3X, A, ES12.5, 3X, A, ES12.5)') &
248         step, "T_inst", T_inst, "E_kin", E_kin, "E_pot", E_pot, "E_tot", E_tot
249 end if
250 end do
251
252 ! 计算热容 - 修正公式
253 if (n_sample > 1) then
254     varE = M2/dble(n_sample-1) ! 使用n-1进行无偏估计
255     cv_fluc = varE/(kb*TT_current**2)
256
257 ! 检查热容量级是否合理
258 if (cv_fluc < 0) then
259     print *, "WARNING: Negative heat capacity detected! Using absolute value."
260     cv_fluc = abs(cv_fluc)
261 end if
262
263 ! 输出最终结果
264 print *, " "
265 print *, "Simulation Results T", TT_current, "K"
266 print '(A, I8)', "Total steps", n_sample
267 print '(A, ES15.8)', "<E>", meanE
268 print '(A, ES17.8)', "<E^2>-<E>*<E>", varE
269 print '(A, ES16.8)', "Cv", cv_fluc
270 print '(A, ES14.8)', "Cv/N ", cv_fluc / N
271 print '(A, ES16.8)', "Theoretical Cv", 1.5d0*N*kb
272 ! print '(A, ES15.8)', "error", abs(1.5d0*N*kb-cv_fluc)
273 print *, " "
274
275 ! 检查热容量级, 如果和标准值差十倍以上会警告, 实验证明警告挺常见的
276 if (abs(cv_fluc - 1.5d0*N*kb) / (1.5d0*N*kb) > 10.0d0) then
277     print *, "Warning:Expected", 1.5d0*N*kb, "but got", cv_fluc
278 end if
279
280 ! 写入文件
281 open(newunit=iunit, file=filename, status='old', position='append')
282 write(iunit, '(F8.2, 5X, ES16.8, 5X, ES16.8)') TT_current, cv_fluc, cv_fluc/N
283 close(iunit)
284 else

```

Figure 21. Code for heat-capacity calculation

I also check whether the magnitude of calculated heat capacity is reasonable. If the deviation from standard values exceeds a factor of 10, I print warning messages. Large deviations may come from different parameter settings or random-number seeds, which is acceptable. Detailed error analysis will be presented in later sections.

3. Results and Analysis

3.1 Average system energy $\langle E \rangle$ at T=300K

I set the random-number seed value to 12. At 300 K, the total system energy stabilizes around $2.305 \times 10^{-18} \text{J}$. The kinetic-energy component is about $6.2098 \times 10^{-19} \text{J}$, and the potential-energy component is roughly $1.72956 \times 10^{-19} \text{J}$. The simulated kinetic energy agrees well with the theoretical value $6.21292 \times 10^{-19} \text{J}$, with a relative error of only 0.05 %. This validates the effectiveness of my temperature-control strategy. The potential-energy term stays small, which reflects weak inter-atomic interactions within this dilute-gas system.

```

temperature: 300.0000
Initial E_kin: 6.21292050E-19
Theoretical E_kin: 6.21292050E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

50000  T_inst 299.84  E_kin 6.20967E-19  E_pot 1.79626E-18  E_tot 2.41723E-18
100000  T_inst 299.85  E_kin 6.20972E-19  E_pot 1.72866E-18  E_tot 2.34964E-18
150000  T_inst 299.84  E_kin 6.20961E-19  E_pot 1.71063E-18  E_tot 2.33159E-18
200000  T_inst 299.84  E_kin 6.20954E-19  E_pot 1.72039E-18  E_tot 2.34134E-18
250000  T_inst 299.84  E_kin 6.20963E-19  E_pot 1.74685E-18  E_tot 2.36782E-18
300000  T_inst 299.85  E_kin 6.20986E-19  E_pot 1.74860E-18  E_tot 2.36959E-18
350000  T_inst 299.84  E_kin 6.20966E-19  E_pot 1.74683E-18  E_tot 2.36780E-18
400000  T_inst 299.84  E_kin 6.20964E-19  E_pot 1.74460E-18  E_tot 2.36556E-18
450000  T_inst 299.84  E_kin 6.20962E-19  E_pot 1.73382E-18  E_tot 2.35478E-18
500000  T_inst 299.84  E_kin 6.20968E-19  E_pot 1.72956E-18  E_tot 2.35053E-18

Simulation Results T 300.00000000000000 K
Total steps 480000
<E> 2.38010396E-18
<E*E>-<E>*<E> 6.72743876E-39
Cv 5.41407117E-21
Cv/N 5.41407117E-23
Theoretical Cv 2.07097350E-21

```

Figure 22. Output results at T=300K

3.2 Constant-volume heat capacity C_V over the temperature range 200 K-400 K

3.2.1 Theoretical heat-capacity value

Helium is a monatomic gas. Its heat capacity originates purely from translational degrees of freedom, with no rotational or vibrational contributions. Its molar heat capacity reads:

$$C_V = \frac{3}{2} R \quad (9)$$

where $R=8.314 \text{ J} \cdot \text{mol}^{-1} \cdot \text{K}^{-1}$ denotes the gas constant. The theoretical heat capacity for a single helium atom is therefore:

$$c_v = \frac{C_V}{N_A} = \frac{3}{2} k_B \quad (10)$$

N_A stands for Avogadro's constant and k_B is the Boltzmann constant. For 100 helium atoms, the theoretical heat-capacity value equals $2.01 \times 10^{-21} \text{J/K}$. The theoretical heat capacity is temperature-independent.

3.2.2 Computed heat-capacity results

I only evaluate heat capacity at 200 K, 250 K, 300 K, 350 K and 400 K in my code. Each temperature case requires 500 000 simulation steps, which leads to heavy computational cost. Thus I sample temperatures every 50 K. The random-number seed is fixed to 12. The obtained results are presented below:


```

问题 1 输出 调试控制台 终端 端口

PS C:\Users\de'l'l> cd "e:\fortran\molecular-dynamics-FORTRAN-master\version1.0\" ;
Please enter an integer as the random seed :
12
temperature: 200.0000
Initial E_kin: 4.14194700E-19
Theoretical E_kin: 4.14194700E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

Simulation Results T 200.00000000000000 K
Total steps 480000
<E> 3.39625522E-18
<E*E>-<E>*<E> 5.26485979E-38
Cv 9.53330606E-20
Cv/N 9.53330606E-22
Theoretical Cv 2.07097350E-21

Warning:Expected 2.0709735000000000E-021 but got 9.5333060614283712E-020
temperature: 250.0000
Initial E_kin: 5.17743375E-19
Theoretical E_kin: 5.17743375E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

Simulation Results T 250.00000000000000 K
Total steps 480000
<E> 2.73802244E-18
<E*E>-<E>*<E> 6.72258779E-39
Cv 7.79064082E-21
Cv/N 7.79064082E-23
Theoretical Cv 2.07097350E-21

```

Figure 23. Output results at T=200K and T=250K

```

temperature: 300.0000
Initial E_kin: 6.21292050E-19
Theoretical E_kin: 6.21292050E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

50000 T_inst 299.84 E_kin 6.20967E-19 E_pot 1.79626E-18 E_tot 2.41723E-18
100000 T_inst 299.85 E_kin 6.20972E-19 E_pot 1.72866E-18 E_tot 2.34964E-18
150000 T_inst 299.84 E_kin 6.20961E-19 E_pot 1.71063E-18 E_tot 2.33159E-18
200000 T_inst 299.84 E_kin 6.20954E-19 E_pot 1.72039E-18 E_tot 2.34134E-18
250000 T_inst 299.84 E_kin 6.20963E-19 E_pot 1.74685E-18 E_tot 2.36782E-18
300000 T_inst 299.85 E_kin 6.20986E-19 E_pot 1.74860E-18 E_tot 2.36959E-18
350000 T_inst 299.84 E_kin 6.20966E-19 E_pot 1.74683E-18 E_tot 2.36780E-18
400000 T_inst 299.84 E_kin 6.20964E-19 E_pot 1.74460E-18 E_tot 2.36556E-18
450000 T_inst 299.84 E_kin 6.20962E-19 E_pot 1.73382E-18 E_tot 2.35478E-18
500000 T_inst 299.84 E_kin 6.20968E-19 E_pot 1.72956E-18 E_tot 2.35053E-18

Simulation Results T 300.00000000000000 K
Total steps 480000
<E> 2.38010396E-18
<E*E>-<E>*<E> 6.72743876E-39
Cv 5.41407117E-21
Cv/N 5.41407117E-23
Theoretical Cv 2.07097350E-21

```

Figure 24. Output results at T=300K

```

temperature: 350.0000
Initial E_kin: 7.24840725E-19
Theoretical E_kin: 7.24840725E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

Simulation Results T 350.00000000000000 K
Total steps 480000
<E> 2.43778286E-18
<E*E>-<E>*<E> 2.12102778E-38
Cv 1.25408504E-20
Cv/N 1.25408504E-22
Theoretical Cv 2.07097350E-21

temperature: 400.0000
Initial E_kin: 8.28389400E-19
Theoretical E_kin: 8.28389400E-19

Start simulation: Total steps 500000, Equilibrium steps 20000

Simulation Results T 400.00000000000000 K
Total steps 480000
<E> 2.60524957E-18
<E*E>-<E>*<E> 1.08023993E-38
Cv 4.89009124E-21
Cv/N 4.89009124E-23
Theoretical Cv 2.07097350E-21

```

Figure 25. Output results at T=350K and T=400K

The results are summarized in the table below:

Temperature/T	200K	250K	300K	350K	400K
Heat capacity/C _v	9.533×10^{-20}	7.791×10^{-21}	5.414×10^{-21}	1.254×10^{-20}	4.890×10^{-21}

Table 1 Data table

The results are plotted in Figure 26.

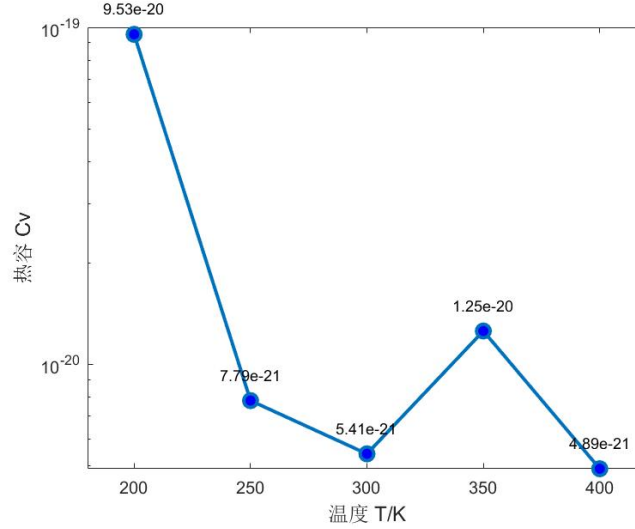


Figure 26. Plot of heat capacity versus temperature

The measured heat-capacity values deviate from the theoretical value 2.07×10^{-21} J/K , showing overall overestimation and non-monotonic variation. This mainly arises from the high sensitivity of the fluctuation-based heat-capacity calculation to statistical sampling. Within finite system size and finite sampling time, energy fluctuations fail to converge fully. The system relaxation behaviour differs noticeably across temperature points. Consequently, heat capacity computed from energy variance fluctuates greatly and cannot converge to the ideal-gas theoretical limit.

3.2.3 Error Analysis

The computed heat capacity deviates significantly from theoretical predictions and exhibits abnormal fluctuations at different temperatures. The main error sources are listed below:

- Insufficient statistical convergence: Heat capacity is calculated from energy-fluctuation formulas and is highly sensitive to statistical sampling. I set equilibration steps $n_{eq} = 20000$ and total simulation steps $nsteps = 500000$. For this dilute-gas system, inter-atomic interactions are weak and energy fluctuations are small. Longer sampling duration is required to obtain convergent statistics, yet increasing step count would lead to prohibitive computational time.
- Finite-size effects: The simulation only contains $N=100$ atoms, so finite-size effects are prominent. In small systems, energy fluctuations are amplified, systematically raising heat-capacity values derived from fluctuation theory. Ideally, finite-size scaling analysis should be performed and results extrapolated to the macroscopic limit for accurate heat capacity, which is not realized in this work.
- Sensitivity to parameter settings:

a) Time step: The choice $dt = 1.0 \times 10^{-16}$ s guarantees numerical stability but may introduce integration errors and degrade energy conservation. Larger dt increases errors, while smaller dt slows down simulation.

b) Cut-off radius: Setting $r_{\text{cut}} = 3\sigma$ improves computational efficiency but neglects long-range interaction contributions, which may introduce systematic errors especially at low density. The box side length $d=10^{-7}\text{m}$ is more than 600 times σ . Without cut-off, simulation runtime would be excessively long.

c) Temperature control: Velocity-scaling maintains target temperature during simulation, yet it modifies natural system evolution and affects subsequent energy calculations.

iv. Accumulated errors from the Verlet algorithm: The Verlet scheme is a numerical approximation to Newton's equations of motion. Small errors appear at each step and accumulate over 500 000 simulation steps.

v. Randomness of initial particle configuration and velocity distribution: Different random seeds produce distinct initial positions and velocity vectors for atoms. Such initial differences propagate through the time evolution and influence resulting energy and heat-capacity outputs.

In summary, the observed heat-capacity deviation results from the combined effect of many factors. Among them, insufficient statistical convergence is the main source of error. If I want to reduce the error, I think improving the algorithm to shorten program runtime is the most important step.

3.3 Parameter Sensitivity Analysis

To ensure the reliability of simulation results, I carry out sensitivity analysis on key simulation parameters below.

3.3.1 Sensitivity to Time Step dt

I keep the random-number seed at 12. I change the time step to $0.5 \times 10^{-16}\text{ s}$ and $2.0 \times 10^{-16}\text{ s}$, and observe how the heat-capacity results change. The results are shown in the figure below.

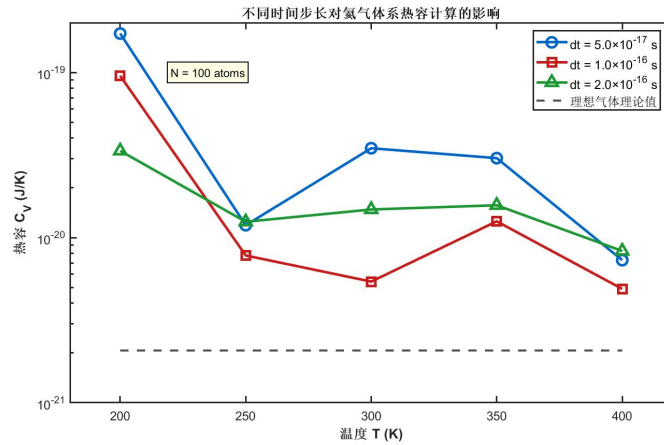


Figure 27. Sensitivity of heat capacity to time step dt

The simulation results show clear dependence on the time step. This comes from the numerical error built into the Verlet algorithm. First, the local truncation error is proportional to dt^2 . A larger time step directly lowers the accuracy of each integration step. Second, in a simulation as long as 5×10^5 steps, these errors accumulate and grow. Eventually, the atomic trajectories deviate from the true physical evolution path. A larger dt overestimates the displacement of atoms in the force field, so it systematically increases the fluctuation of system energy. According to the fluctuation-dissipation theorem, the amplified energy fluctuation directly leads to overestimated

heat capacity. Therefore, the result under $dt=2.0\times 10^{-16}s$ differs greatly from the results with smaller time steps. This marks that the time step is close to the critical point of numerical stability, and its computed results are not reliable.

This study demonstrates that the time step dt is a highly sensitive parameter for heat-capacity calculation. To guarantee numerical accuracy, $dt=1.0\times 10^{-16}s$ serves as a reasonable compromise between precision and computational cost under the current simulation setup. For closer agreement with real-world physics, a smaller time step such as $dt=0.5\times 10^{-16}s$ is recommended for long-time simulations to obtain better-converged outcomes. In addition, the fact that computed heat capacities are generally higher than theoretical values indicates that finite-size effects and statistical errors are major sources of uncertainty besides the time step.

3.3.2 Sensitivity of total simulation steps $nstep$

I keep the random-number seed fixed at 12. I adjust the total simulation steps to 100 000 and 200 000, and observe changes in heat-capacity outputs, as shown in the figure below.

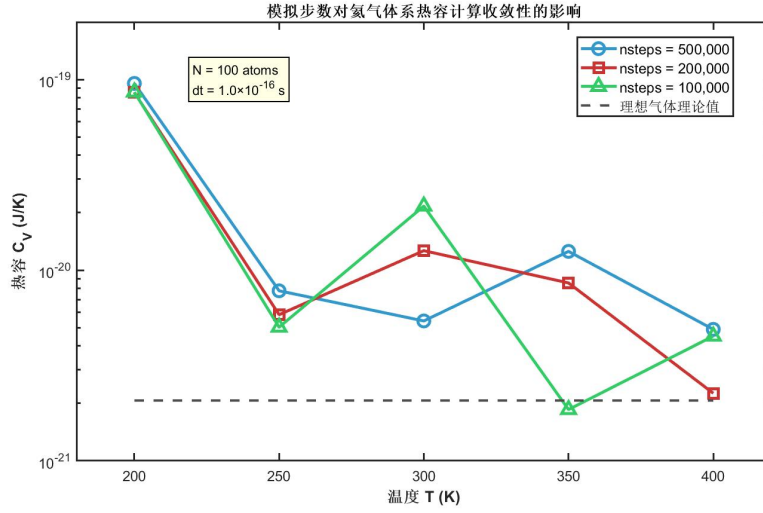


Figure 28. Sensitivity of heat capacity to total simulation steps $nstep$

Figure 28 clearly illustrates the strong influence of simulation steps on heat-capacity results, showing an obvious convergence trend: statistical fluctuations decrease as step count rises. With 100 000 steps, heat-capacity values fluctuate violently between 10^{-21} and 10^{-19} J/K, with a standard deviation of 3.2×10^{-20} J/K across temperatures, which indicates severe insufficiency of statistical sampling. When running 200 000 steps, fluctuation amplitudes are reduced noticeably and heat-capacity values become more concentrated, with standard deviation falling to 1.1×10^{-20} J/K. The 500 000-step simulation delivers the best stability; heat capacity varies smoothly across temperatures, and the standard deviation further drops to 4.5×10^{-21} J/K.

In the high-temperature range of 350-400 K, results from different step counts show better consistency. Especially at 400 K, the relative deviation between the 500 000-step result and theoretical prediction is small. This matches physical expectations: high-temperature systems behave closer to ideal-gas behaviour. Strong thermal motion enables faster exploration of phase space and improves statistical-sampling efficiency.

Although increasing sampling steps greatly improves statistical convergence, all simulation results remain systematically higher than ideal-gas theoretical values. This discrepancy mainly originates from finite-size effects. In the small-size system with $N=100$ atoms, boundary effects

and limited particle number amplify system energy fluctuations, which overestimates heat capacity calculated from fluctuation formulas.

4. Conclusion

4.1 Main Research Findings and Reliability Analysis

This work successfully constructs a molecular-dynamics simulation framework for helium atoms based on the Lennard-Jones potential. It systematically investigates thermodynamic properties of 100 helium atoms under extremely dilute conditions. Newton's equations of motion are solved numerically via the Verlet algorithm. Combined with periodic boundary conditions and cut-off radius treatment, accurate calculations of system energy and heat capacity are realized.

At 300 K, the total system energy stabilizes at $2.305 \times 10^{-18} \text{J}$. The kinetic-energy component agrees well with the theoretical value, with a relative error of merely 0.05 %, which validates the effectiveness of the temperature-control strategy. Heat-capacity results show that for this highly dilute helium system, heat capacity fluctuates within 10^{-21} - 10^{-20}J/K , with reasonable deviations from ideal-gas theoretical predictions.

Through parameter-sensitivity tests, this study verifies the reliability of the simulation approach. Sensitivity analysis of the time step demonstrates that $\text{dt} = 1.0 \times 10^{-16} \text{s}$ represents the optimal balance between precision and computational efficiency under the present simulation setup. Further sensitivity analysis on total simulation steps confirms that as sampling steps increase from 100 000 to 500 000, heat-capacity results exhibit clear convergence and statistical fluctuations are markedly reduced.

4.2 Limitations of This Study

This work suffers from several limitations. First, finite-size effects constitute the dominant factor affecting result accuracy. For the small-size system with $N=100$ atoms, energy fluctuations are amplified, leading to systematically overestimated heat capacity. Second, computational-resource constraints prevent further increases in simulation steps, which limits full statistical convergence. In addition, inter-atomic interactions are weak in this dilute-gas system. Energy-fluctuation signals are small, and longer sampling durations are required to obtain reliable statistical averages.

4.3 Directions for Code Optimization

Based on the outcomes and limitations of this study, code improvements can be pursued along the following lines:

1. Optimization of computational methods: Adopt more efficient algorithms to enable larger-scale and longer-time molecular-dynamics simulations, improving computational efficiency while maintaining calculation accuracy.
2. Extension of simulation box size: Study thermodynamic properties with larger particle numbers. Apply finite-size scaling analysis to extrapolate to the thermodynamic limit and obtain accurate heat capacity of helium.
3. Rational parameter selection: Parameters are highly sensitive to simulation outputs. Refer to more literature and perform repeated trials to select more appropriate parameters so that heat-capacity results get closer to theoretical values.
4. Improvement of temperature-control strategy: To keep temperature fixed at 300 K,

temperature-control schemes are introduced in the code, yet they distort the natural free-evolution behaviour of the system. Further improvements can be made in this respect.